SJ INNOVATION · INTERN ASSIGNMENT

# QuickBite — Food Ordering & Delivery App

Vibe-coding assignment — Lovable + Supabase + GitHub + Vercel

| | |
|---|---|
| **Intern** | Sanket (Goa) |
| **Mentor** | Amol — Tech Manager |
| **Issued** | Tuesday, 16 September 2026 |
| **Demo due** | **Wednesday, 23 September 2026** (live demo, screen-share) |

> **Your mission this week:** Build a food-delivery app like Swiggy/Zomato in miniature. A **Customer** browses restaurants and places an order, a **Restaurant Manager** accepts the order and prepares the food, and a **Delivery Rider** picks it up and delivers it. The interesting part is the **handoff chain**: one order moves through three different people, each seeing only their own step.

## 1. New skills you will learn this week

This project is not just about "building an app". By the end you should be able to explain these four ideas to someone else:

### Vibe coding (with Lovable)

**Vibe coding** means you build software by *describing what you want in plain English* instead of writing every line of code by hand. You chat with an AI builder, it writes and edits the code and shows you a live preview, you look at the result, and you refine your request. **Lovable** ( `lovable.dev` ) is the tool we use for this. Your job is to give clear, specific instructions and to review what the AI produced — the AI is your junior developer, you are the manager.

### Supabase (the backend & database)

**Supabase** is the "backend" that stores your data and handles logins. It gives you three things you will use: a **PostgreSQL database** (tables with rows and columns), **Authentication** (sign-up / login with email & password or Google), and **Row Level Security (RLS)** — rules that decide which user is allowed to see or change which row. RLS is how three different roles safely share one database.

### GitHub (where the code lives)

**GitHub** stores your project's code in a "repository" (repo). Lovable can push your code to GitHub automatically, so there is always a safe, versioned copy. This is also the bridge to deployment.

### Vercel (making it live on the internet)

**Vercel** takes the code from your GitHub repo and publishes it at a real, shareable URL (e.g. `your-project.vercel.app` ). This is the link you will demo. You will deploy using **your own Vercel account**.

## 2. Accounts & one-time setup

Use the shared **SG Notion email ID (free)** for Lovable, Supabase and GitHub. Use **your own account** for Vercel. Set all of this up on Day 1.

| Tool | Account to use | What to do |
|---|---|---|
| **Lovable** | SG Notion email (free plan) | Sign up at `lovable.dev` . The free plan gives roughly **5 credits per day** that reset daily, and projects are public — that is fine for this assignment. |
| **Supabase** | SG Notion email (free) | Sign up at `supabase.com` , create one new project, and note the **Project URL** and **anon key** (Project Settings → API). |
| **GitHub** | SG Notion email | Create/sign in to the GitHub account. In Lovable, connect GitHub so your project can be pushed to a repo. |
| **Vercel** | **Your own account** | Sign up at `vercel.com` — you can log in *with your GitHub account* so it can read your repo. |

> **Connect Lovable to Supabase (do this once):** In the Lovable editor open `More → Cloud` (or the Supabase icon), choose **"Connect Supabase"**, authorise it, and select the Supabase project you created. After this, when you ask Lovable to store data it will create the tables inside *your* Supabase project and show you the SQL migration to approve before it runs.

## 3. The three roles & access levels

Every logged-in user has exactly one role. The role decides what they can see and do.

| Role | Access level |
|---|---|
| Customer | Orders food. Sees restaurants, their own cart and their own orders only. |
| Restaurant Manager | Runs a restaurant. Manages only their own menu and the orders for their restaurant. |
| Delivery Rider | Delivers orders. Sees deliveries that are ready for pickup and their own active deliveries. |

## 4. Sidebar menu for each role

After login, each role must see a **different left-hand sidebar**. This is the most visible sign that role-based access works.

| Role | Left sidebar menu items |
|---|---|
| **Customer** | Browse Restaurants, My Cart, My Orders, Track Order, My Profile |
| **Restaurant Manager** | Restaurant Dashboard, My Menu (add/edit items), Incoming Orders, Order History, My Profile |
| **Delivery Rider** | Rider Dashboard, Available Deliveries, My Deliveries, Delivery History, My Profile |

## 5. How the roles work together (the workflow)

The whole point is that the three roles *hand work to each other*. Build this end-to-end flow:

1. A **Restaurant Manager** adds menu items and marks the restaurant open.
2. A **Customer** browses restaurants, adds items to the cart and places an order → status **Placed**.
3. The **Restaurant Manager** sees the incoming order, **Accepts** it (status **Preparing**), then marks it **Ready for pickup**.
4. A **Delivery Rider** sees the order under Available Deliveries and **picks it up** → status **Out for delivery**.
5. The Rider marks the order **Delivered**.
6. The **Customer** tracks the status live and can **rate** the order after delivery.

## 6. Database design in Supabase

Create these tables in your Supabase project. Follow the naming standards below.

| Table | Columns | Purpose |
|---|---|---|
| `profiles` | id (uuid, = auth user), full_name, email, role (customer / restaurant / rider), phone, created_at | One row per user; holds the role. |
| `restaurants` | id, owner_id → profiles, name, cuisine, address, is_open (boolean), created_at | A restaurant, owned by a restaurant-manager user. |
| `menu_items` | id, restaurant_id → restaurants, name, description, price (numeric), is_available (boolean), image_url, created_at | Dishes on a menu. |
| `orders` | id, customer_id → profiles, restaurant_id → restaurants, rider_id → profiles (nullable), total_amount (numeric), status (placed / preparing / ready / out_for_delivery / delivered / cancelled), delivery_address, created_at | One food order moving through the chain. |
| `order_items` | id, order_id → orders, menu_item_id → menu_items, quantity, unit_price (numeric) | Line items inside an order. |
| `ratings` | id, order_id → orders, customer_id → profiles, restaurant_id → restaurants, stars (int 1-5), comment, created_at | Customer's rating after delivery. |

### Naming standards (follow these exactly)

- Table names: **lower_case, plural, snake_case** — `order_items`, not `OrderItem`.
- Column names: **snake_case** — `created_at`, `full_name`.
- Every table has an `id` primary key and a `created_at` timestamp.
- A column that points to another table ends in `_id` and is a **foreign key** — e.g. `product_id` references `products.id`.
- Fixed choice values (like status) use clear lowercase words, e.g. `pending / approved`.

## 8. Login & authentication (required)

Users must be able to **sign up and log in**. Implement **both** of these:

- **Email & password** login (ask Lovable: "*Add email and password sign-up and login, and require users to be logged in to see the dashboard*").
- **Google (social) login** — enable the Google provider in Supabase → Authentication, then ask Lovable to "*Add a Sign in with Google button on the login page*".

On sign-up, default new users to the **customer** role. Let a user register as a restaurant or rider (a role choice on sign-up), or set those roles manually in Supabase. The rider_id on an order stays empty until a rider picks it up.

> **Role-based access is the heart of this project.** A logged-in user must only see the sidebar and data for *their* role. Store the role in a `profiles` table and use **Supabase Row Level Security** so, for example, one customer can never read another customer's orders.

## 7. Making the most of 5 credits/day

On the free plan every message to Lovable costs a credit, and you only get about **5 per day**. Momentum comes from **planning your prompts before you type them**, not from firing off small requests. Rules of thumb:

- **Plan on paper first.** Write down what you want built *today* before opening Lovable. One good, detailed prompt beats five vague ones.
- **Batch related requests** into a single message (e.g. "Create the products page *and* the product detail page *and* add them to the seller sidebar").
- **Be specific**: name the page, the fields, the role that should see it, and the table it reads/writes.
- **Use the visual editor** for tiny tweaks (text, colours, spacing) — those do not need a credit.
- **Fix, don't restart.** If something is wrong, describe the exact problem instead of re-generating the whole app.

Examples of well-formed prompts for this project:

```
Create a 'Browse Restaurants' page for customers showing restaurants where is_open is true; clicking one shows its available menu_items with an 'Add to cart' button. Reading from Supabase.
```

```
Add a Restaurant Dashboard with a sidebar (My Menu, Incoming Orders). On 'Incoming Orders' show orders for this restaurant with status 'placed', an Accept button (sets status 'preparing') and a 'Ready' button (sets status 'ready').
```

```
Add a Rider 'Available Deliveries' page listing orders with status 'ready' and no rider yet; a 'Pick up' button sets rider_id to the logged-in rider and status to 'out_for_delivery'.
```

## 9. Day-by-day plan (7 days)

You get about 5 Lovable credits/day, so this plan spreads the work. Do a little every day — do not leave it for the weekend.

| Day | Goal |
|---|---|
| **Day 1 (Wed)** | Set up all 4 accounts. Connect Lovable→Supabase→GitHub. Add email/password + Google login. Create the profiles table with role. |
| **Day 2 (Thu)** | Role-based routing: customer / restaurant / rider each land on their own dashboard with their own sidebar. |
| **Day 3 (Fri)** | Restaurants + menu_items tables. Restaurant Manager can add/edit menu items. Customer 'Browse Restaurants' + menu. |
| **Day 4 (Sat)** | Cart + place order: orders, order_items. Customer places an order (status 'placed'). |
| **Day 5 (Sun)** | Restaurant accepts/prepares/ready; Rider picks up and delivers. 'Track Order' page for the customer. |
| **Day 6 (Mon)** | Ratings + Row Level Security: customers see only their own orders; restaurants only their orders; riders only their deliveries. Polish sidebars. |
| **Day 7 (Tue)** | Deploy to Vercel, test all 3 logins on the live URL, fill the submission report. Buffer for fixes. |

## 10. Push to GitHub

1. In the Lovable editor, open the **GitHub** option (top-right menu).
2. Click **Connect to GitHub** and authorise the SG Notion GitHub account.
3. Choose **Create Repository** — Lovable creates a repo and pushes your code. Every later change syncs automatically.
4. Confirm you can see the repo on `github.com` with the code inside.

## 11. Deploy to Vercel (your own account)

1. Log in to `vercel.com` with your GitHub account.
2. Click **Add New → Project** and **Import** the GitHub repo Lovable created.
3. Vercel auto-detects the framework (Vite/React). Leave build settings as default.
4. Open **Environment Variables** and add your Supabase keys so the live site can reach the database:
   - `VITE_SUPABASE_URL` = your Supabase Project URL
   - `VITE_SUPABASE_ANON_KEY` = your Supabase anon/public key

   (Use the exact variable names shown in your Lovable project's code / `.env` file.)
5. Click **Deploy**. After a minute you get a live URL like `your-project.vercel.app` — that is your demo link.
6. Add Vercel's URL to Supabase → Authentication → URL Configuration (Site URL + Redirect URLs) so Google login works on the live site.

## 13. How you will be evaluated (Wednesday demo)

You will screen-share and walk through the live Vercel link. We will log in as each of the three roles. You are marked on:

| Checkpoint |
|---|
| Email/password AND Google login both work on the live Vercel site. |
| Customer, Restaurant Manager and Delivery Rider each see a clearly different sidebar. |
| A single order moves correctly through Placed → Preparing → Ready → Out for delivery → Delivered. |
| The right restaurant sees the order, and a rider can pick it up and deliver it. |
| A customer cannot see another customer's orders (RLS works). |
| Sensible table & column names in Supabase (snake_case, foreign keys correct). |

> **Deliverables due by the demo:** (1) Live Vercel URL, (2) GitHub repo link, (3) Lovable project link, (4) Supabase project, (5) working login for all three roles with test credentials, filled into the **Demo Submission & Test Report**.